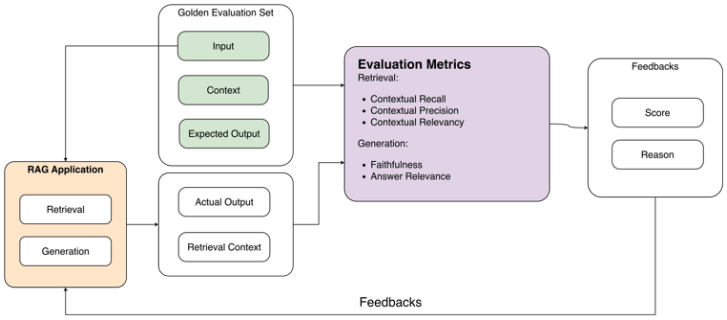
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Synthetic Data Generation for evaluating RAG

1. Introduction

Evaluating RAG (Retrieval-Augmented Generation) augmented pipelines is crucial for assessing their performance.



However, manually creating hundreds of QA (Input-Context-Expected Output) samples from documents can be **time-consuming and labor-intensive**. Additionally, human-generated questions may struggle to reach the level of complexity required for a thorough evaluation, ultimately impacting the quality of the assessment.

Synthetic data generation **leverages LLMs to create quality data without the need to manually collect, clean, and annotate massive datasets**. With models like Sonet, Gemini, or GPTs, it's now possible to synthetically produce datasets that are **more comprehensive and diverse than human-labeled ones**, in far less time, which can be used to benchmark LLM (systems)

By using synthetic data generation developer time in data aggregation process can be reduced by 90%.

- 📐 Synthetic Test Data generation | Ragas
- Using LLMs for Synthetic Data Generation: The Definitive Guide - Confident AI

2. What is Synthetic Data Generation (Leveraging capabilities of LLMs a.k.a LLM-as-a-judge) ?

Synthetic data generation using LLMs involves using an LLM to **create artificial data**, which often are datasets that can be used to train, fine-tune, and evaluate LLMs themselves. **Generating synthetic datasets is not only faster than scouring public datasets and cheaper than human annotation but also results in higher quality and data diversity.**

When it comes to generating a synthetic dataset for evaluation, there are two main methods: self-improvement from using your model's output, or distillation from a more advanced model.

- **Self-improvement:** involves your model generating data iteratively from its own output without external dependencies
- **Distillation:** involves using a stronger model to generate synthetic data for to evaluate a weaker model

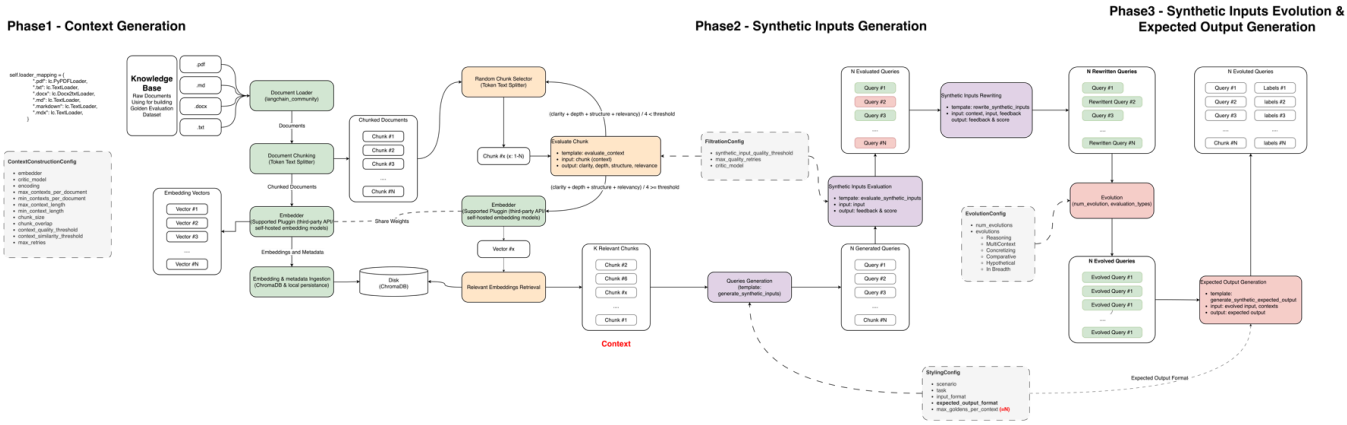
Self-improvement methods, such as Self-Instruct: Aligning Language Models with Self-Generated Instructions or SPIN (Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models), are limited by a model's capabilities and may suffer from amplified biases and errors. In contrast, distillation techniques are only limited by the best model available, ensuring the highest quality generation.

3. Generating Data from your Knowledge Base

The process starts with the creation of synthetic queries, which are generated using context from your knowledge base (often in the form of documents) as the ground truth. The generated queries are then "evolved" multiple times to complicate and make realistic, and when combined with the original context it was generated from, makes up your final synthetic dataset. Although optional, you can also choose to generate a target label for each synthetic query-context pair, which will act as the expected output of your LLM system for a given query.

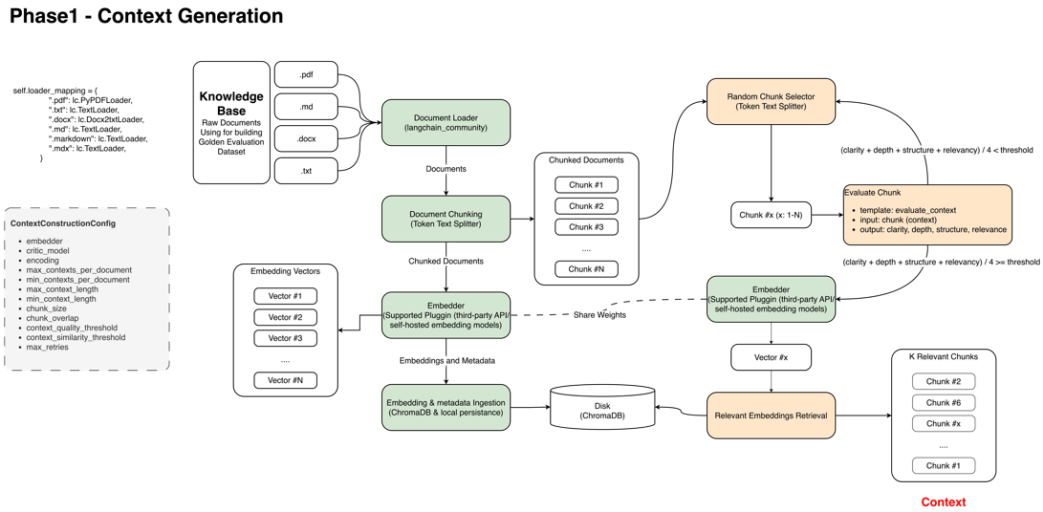
@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

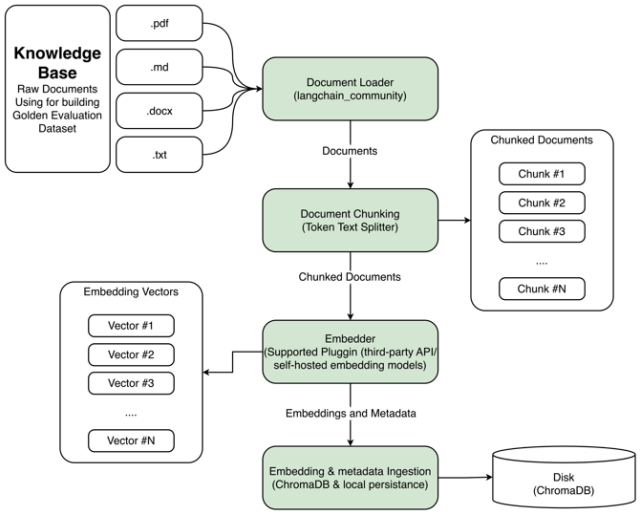


Phase 1 - Context Generation

During context generation, a document — or multiple documents — from your **knowledge base** is divided into chunks using a token splitter. A random chunk is then selected, and additional chunks are retrieved and grouped with the selected one based on their **similarity**.



Context Vector Storage



Data Sourcing (Knowledge Base)

The process begins with the **Knowledge Base**, which consists of raw documents used for building evaluation datasets. Supported file formats include:

@Copyright 2025 by Vietnam Silicon.

- `.pdf`, `.md` (Markdown), `.docx`, and `.txt`.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Document Loading & Processing

- **Document Loader:** Uses libraries like `langchain_community` to ingest the raw files and convert them into a standardized document format.
- **Document Chunking:** The documents are passed to a **Token Text Splitter**. This breaks long texts into smaller, manageable "chunks" (Chunk #1 to Chunk #N). This is crucial for staying within the context limits of LLMs.

Embedding Generation

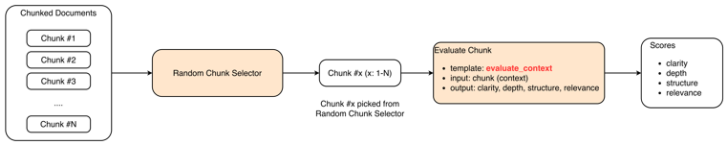
- **Embedder:** The chunked documents are sent to an embedding model (via third-party APIs or self-hosted models).
- **Embedding Vectors:** The text chunks are transformed into numerical **Embedding Vectors**. These vectors represent the semantic meaning of the text in a high-dimensional space.

Storage and Persistence

- **Ingestion:** The generated embeddings, along with their associated metadata (like source name or page number), are ingested into a vector database.
- **Vector Database (ChromaDB):** The diagram specifies **ChromaDB** with local persistence. This "Disk" storage allows the system to efficiently retrieve relevant document chunks later by performing a similarity search against the saved vectors.

Context Filtering

During context generation, there's a chance you might randomly select a low-quality chunk. Oftentimes, your knowledge base may contain complex structures or excess whitespace that becomes unintelligible when broken down. Employing LLMs as judges is a robust method for identifying and eliminating these low-quality contexts.

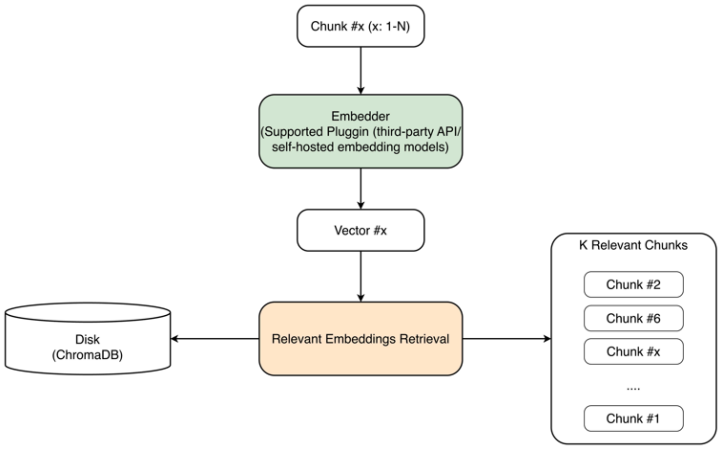


Each chunk is randomly sampled for each context and scored based on the following criteria:

- **Clarity:** How clear and understandable the information is.
- **Depth:** The level of detail and insight provided.
- **Structure:** How well-organized and logical the content is.
- **Relevance:** How closely the content relates to the main topic.

Relevant Context Retrieval

A random **qualified** chunk is then embedded, and additional chunks are retrieved and grouped with the selected one based on their **similarity** from vector storage.

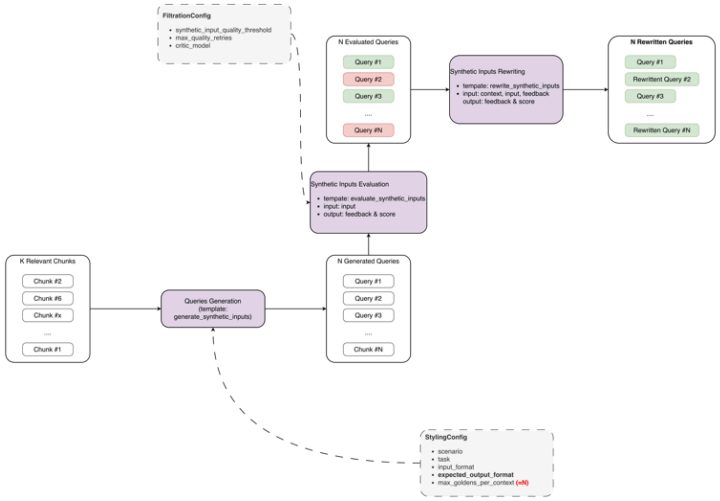


Phase 2 - Synthetic Inputs Generation

@Copyright 2025 by Vietnam Silicon.

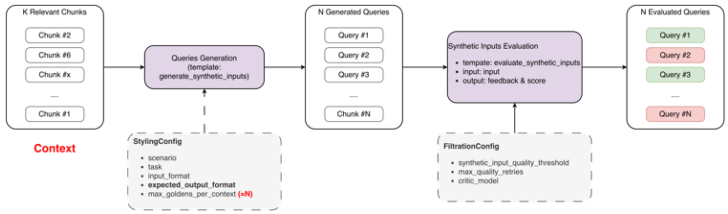
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Phase2 - Synthetic Inputs Generation



Styling Synthetic Data

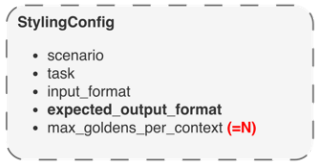
In the next stage, **synthetic inputs** are generated from the **goldens (relevant chunks)**



Finally, you may want to **tailor your queries to specific topics and customize their input and output formats** to fit your unique use cases.

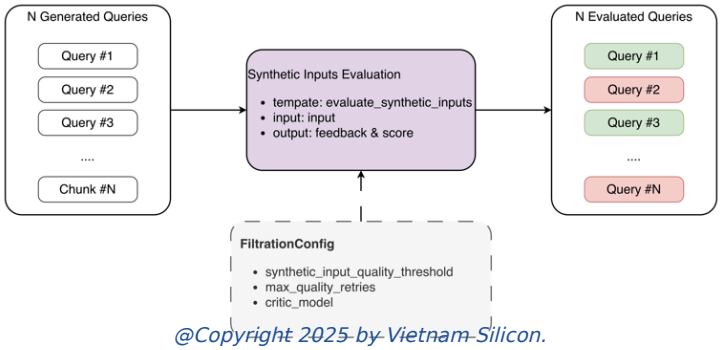
You should plan to apply specific styling *during the initial generation, through any evolutionary changes, and after the final outputs are generated*. Revisiting the style after the initial generation is crucial, as the evolution of synthetic queries can alter the initial styling intentions.

The extent of style adjustments after the first round will depend on your desired level of control over the final product and the associated cost implications.



Synthetic Input Evaluation (Filtering synthetic inputs)

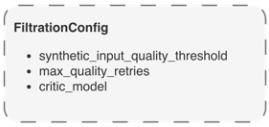
Step of filtering **synthetic inputs generated from these contexts** is crucial because even well-curated contexts can sometimes lead to the generation of inputs that *might not meet the required standards*



@Copyright 2025 by Vietnam Silicon.

This step conducted by adopting LLM as a judge with prompt template (evaluate_synthetic_inputs)

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

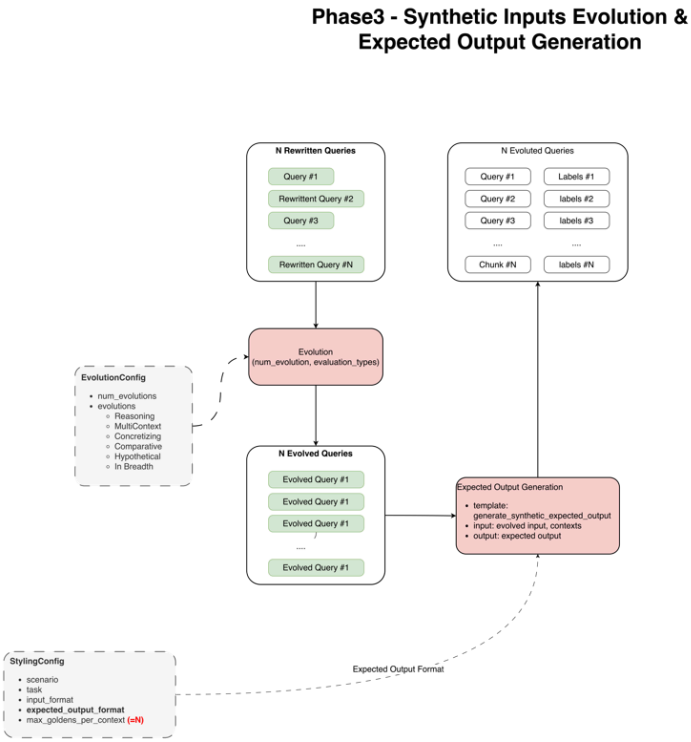


Here are a few criteria you may want to judge your synthetic input on:

- **Self-containment:** Ensures the input is complete and can function independently without external references.
- **Clarity:** Checks that the input clearly communicates its intended message or question to avoid misinterpretation.
- **Consistency:** Assures the input aligns with the provided context or background information thematically and factually.
- **Relevance:** Verifies that the input directly pertains to the intended tasks or queries, ensuring it's purposeful and on-topic.
- **Completeness:** Confirms the input includes all necessary details for effective interaction or query resolution.

Using these criteria helps ensure that synthetic inputs are not only high in quality but also tailored perfectly to their intended applications.

Phase 3 - Evolution



In the **evolution** step, the `input`s of the filtered goldens are rewritten to make **more complex and realistic**, often times indistinguishable from human curated goldens. Each `input` is rewritten `num_evolutions` times, where each evolution is sampled from the `evolution` distribution which adds an additional layer of complexity to the rewritten `input`.

DeepEval offers 7 types of evolutions: reasoning, multicontext, concretizing, constrained, comparative, hypothetical, and in-breadth evolutions.

- **Reasoning:** Evolves the input to require multi-step logical thinking.
- **Multicontext:** Ensures that all relevant information from the context is utilized.
- **Concretizing:** Makes abstract ideas more concrete and detailed.
- **Constrained:** Introduces a condition or restriction, testing the model's ability to operate within specific limits.
- **Comparative:** Requires a response that involves a comparison between options or contexts.
- **Hypothetical:** Forces the model to consider and respond to a hypothetical scenario.
- **In-breadth:** Broadens the input to touch on related or adjacent topics.

Best Practices for Using Evolutions

@Copyright 2025 by Vietnam Silicon.

To maximize the effectiveness of evolutions in your testing process, consider the following best practices:

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

1. **Align Evolutions with Testing Goals**: Choose evolutions based on what you're trying to evaluate. For reasoning or logic tests, prioritize evolutions like Reasoning and Comparative. For broader domain testing, increase the use of In-breadth evolutions.
2. **Balance Complexity and Coverage**: Use a mix of vertical complexity (e.g., Reasoning, Constrained) and horizontal expansion (e.g., In-breadth) to ensure a comprehensive evaluation of both deep reasoning and a broad range of topics.
3. **Start Small, Then Scale**: Begin with a smaller number of evolution steps (`num_evolutions`) and gradually increase complexity. This helps you control the challenge level without generating overly complex goldens.
4. **Target Edge Cases for Stress Testing**: To uncover edge cases, increase the use of Constrained and Hypothetical evolutions. These evolutions are ideal for testing your model under restrictive or unusual conditions.
5. **Monitor Evolution Distribution**: Regularly check the distribution of evolutions to avoid overloading test data with any single type. Maintain a balanced distribution unless you're focusing on a specific evaluation area.

3. Metrics

Faithfulness

- measures the factual consistency of `the generated answer` (actual output) against the `given context` (retrieval context). It is calculated from answer and retrieved context.

Logic Flow (mermaid)

```
flowchart TD
    Start([measure called]) --> CheckParams[Check LLM test case params<br/>INPUT, ACTUAL_OUTPUT &<br/>RETRIEVAL_CONTEXT required]
    CheckParams --> AsyncCheck{async_mode?}

    AsyncCheck -->|Yes| AsyncMeasure[Run a_measure via event loop]
    AsyncCheck -->|No| SyncFlow[Continue synchronous flow]

    %% Parallel Generation in Async Mode
    AsyncMeasure --> ParallelGen[Generate Truths & Claims in parallel<br/>using asyncio.gather]
    ParallelGen --> Step1A[Generate Truths async]
    ParallelGen --> Step2A[Generate Claims async]

    %% Synchronous Flow
    SyncFlow --> Step1[Generate Truths]
    SyncFlow --> Step2[Generate Claims]

    %% Generate Truths Detail
    Step1 --> Prompt1["<b>PROMPT 1: generate_truths</b><br/>---<br/>Input: retrieval_context<br/>Task: Extract factual truths from context<br/>---<br/>Template Structure:<br/>- Extraction limit handling:<br/>  • None: all truths<br/>  • 1: single most important<br/>  • N: N most important per doc<br/>- Instructions for coherent truths<br/>- Example with Einstein/Nobel Prize<br/>- Multimodal instructions if applicable<br/>- JSON format requirement<br/>- Note: factual but correctness not validated<br/>---<br/>Output: List of truth statements"]
    Step1A --> Prompt1

    Prompt1 --> LLMCall1[LLM extracts truths from context]
    LLMCall1 --> Extract1[Extract: truths list]
    Extract1 --> StoreTruths[Store in self.truths]

    %% Generate Claims Detail
    Step2 --> Prompt2["<b>PROMPT 2: generate_claims</b><br/>---<br/>Input: actual_output<br/>Task: Extract factual claims from AI output<br/>---<br/>Template Structure:<br/>- Extract FACTUAL claims<br/>- Claims must be coherent, not out of context<br/>- Include full context, not cherry-picked<br/>- Example with Einstein/Nobel Prize<br/>- Multimodal: extract from text AND images<br/>- JSON format requirement<br/>- Note: claims should be factual statements<br/>  (correctness doesn't matter)<br/>- Take text at face value, no prior knowledge<br/>---<br/>Output: List of claim statements"]
    Step2A --> Prompt2

    Prompt2 --> LLMCall2[LLM extracts claims from output]
    LLMCall2 --> Extract2[Extract: claims list]
    Extract2 --> StoreClaims[Store in self.claims]

    %% Convergence Point
    StoreTruths --> Converge[Both truths & claims ready]
    StoreClaims --> Converge

    %% Generate Verdicts Detail
    Converge --> CheckEmpty{claims<br/>empty?}
    CheckEmpty -->|Yes| EmptyVerdicts[verdicts = empty list]
    CheckEmpty -->|No| Step3[Generate Verdicts]
```

@Copyright 2025 by Vietnam Silicon.

```
    Step3 --> Prompt3["<b>PROMPT 3: generate_verdicts</b><br/>---<br/>Input: claims + retrieval_context (truths)<br/>Task: Check if claims contradict context<br/>---<br/>Template Structure:<br/>- Verdict options: yes/no/idk<br/>- 'yes' = claim aligns with
```

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

```
context<br/>- 'no' = DIRECT contradiction (provide reason)<br/>- 'idk' = not backed up / no info (provide reason)<br/>- Guidelines: <br/>  • ONE verdict per claim<br/>  • Only 'no' if DIRECT contradiction<br/>  • Never use prior knowledge<br/>  • Vague language (may/possibly) ≠ contradiction<br/>  • Missing info → 'idk', not 'no'<br/>- Multimodal: includes example verdicts<br/>- JSON format with verdicts array<br/>---<br/>Output: List of FaithfulnessVerdict"]

    Prompt3 --> LLMCall3[LLM evaluates each claim<br/>against truths]
    LLMCall3 --> Extract3[Extract: list of verdicts with reasons]
    Extract3 --> StoreVerdicts[Store in self.verdicts]
    EmptyVerdicts --> StoreVerdicts

    %% Calculate Score
    StoreVerdicts --> CalcScore[Calculate Score]
    CalcScore --> ScoreLogic["Score Calculation:<br/>---<br/>If verdicts empty: score = 1<br/>Else:<br/>  faithfulness_count = verdicts != 'no'<br/>  If penalize_ambiguous_claims:<br/>  faithfulness_count -= count of 'idk'<br/>  score = faithfulness_count / total_verdicts<br/>If strict_mode AND score < threshold:<br/>  score = 0"]

    %% Generate Reason Detail
    ScoreLogic --> Step4[Generate Reason]
    Step4 --> CheckIncludeReason{include_reason?}
    CheckIncludeReason -->|No| NoReason[reason = None]
    CheckIncludeReason -->|Yes| CollectContradictions[Collect contradictions<br/>where verdict = 'no']

    CollectContradictions --> Prompt4["<b>PROMPT 4: generate_reason</b><br/>---<br/>Input: contradictions + score<br/>Task: Summarize why score is what it is<br/>---<br/>Template Structure:<br/>- Explain contradictions list context<br/>- Summarize contradictions to justify score<br/>- If no contradictions:<br/>  positive upbeat message<br/>- Must use info from contradictions<br/>- Be definitive (act like you know actual output)<br/>- JSON format with 'reason' key<br/>---<br/>Output: Explanation string"]

    Prompt4 --> LLMCall4[LLM generates explanation]
    LLMCall4 --> Extract4[Extract: reason string]
    Extract4 --> StoreReason[Store in self.reason]
    NoReason --> StoreReason

    %% Final Steps
    StoreReason --> CheckSuccess[Determine success<br/>success = score >= threshold]
    CheckSuccess --> VerboseLogs[Construct verbose logs:<br/>1. Truths with extraction limit<br/>2. Claims list<br/>3. Verdicts list<br/>4. Score + Reason]
    VerboseLogs --> PostMetric{log_metric_to<br/>_confident?}
    PostMetric -->|Yes| SendMetric[Post metric data]
    PostMetric -->|No| ReturnScore
    SendMetric --> ReturnScore([Return self.score])

    %% Styling
    classDef promptClass fill:#e1f5ff,stroke:#0066cc,stroke-width:2px
    classDef llmClass fill:#fff4e1,stroke:#ff9900,stroke-width:2px
    classDef calcClass fill:#f0e1ff,stroke:#9900cc,stroke-width:2px
    classDef parallelClass fill:#e1ffe1,stroke:#00cc00,stroke-width:2px

    class Prompt1,Prompt2,Prompt3,Prompt4 promptClass
    class LLMCall1,LLMCall2,LLMCall3,LLMCall4 llmClass
    class CalcScore,ScoreLogic calcClass
    class ParallelGen,Converge parallelClass

```

Answer Relevance

- focuses on assessing how pertinent the `generated answer` (actual output) is to the `given prompt` (input). A lower score is assigned to answers that are incomplete or contain redundant information and higher scores indicate better relevancy. This metric is computed using the `question` (input), the `context` (retrieval context) and the `answer` (actual output).

Logic Flow (mermaid)

```
flowchart TD
    Start([measure called]) --> CheckParams[Check LLM test case params<br/>INPUT & ACTUAL_OUTPUT required]
    CheckParams --> AsyncCheck{async_mode?}

    AsyncCheck -->|Yes| AsyncMeasure[Run a_measure via event loop]
    AsyncCheck -->|No| SyncFlow[Continue synchronous flow]

    SyncFlow --> Step1[Generate Statements]
    AsyncMeasure --> Step1A[Generate Statements async]

    %% Generate Statements Detail
```

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

```
    Step1 --> Prompt1["<b>PROMPT 1: generate_statements</b><br/>---<br/>Input: actual_output<br/>Task: Break down output into list of statements<br/>---<br/>Template Structure:<br/>- Instructions for breakdown<br/>- Example with laptop features<br/>- Multimodal rules if applicable<br/>- JSON format requirement<br/>---<br/>Output: List of statements"]
    Step1A --> Prompt1

    Prompt1 --> LLMCall1[LLM generates structured response]
    LLMCall1 --> Extract1[Extract: statements list + MLLMImages]
    Extract1 --> StoreStatements[Store in self.statements]

    %% Generate Verdicts Detail
    StoreStatements --> CheckEmpty{statements<br/>empty?}
    CheckEmpty -->|Yes| EmptyVerdicts[verdicts = empty list]
    CheckEmpty -->|No| Step2[Generate Verdicts]

    Step2 --> Prompt2["<b>PROMPT 2: generate_verdicts</b><br/>---<br/>Input: input + statements<br/>Task: Determine relevance of each statement<br/>---<br/>Template Structure:<br/>- Relevance evaluation instructions<br/>- Verdict options: yes/no/idk<br/>- 'yes' = relevant<br/>- 'no' = irrelevant<br/>- 'idk' = ambiguous/supporting<br/>- Reason required for 'no' and 'idk'<br/>- Multimodal rules if applicable<br/>- JSON format with verdicts array<br/>---<br/>Output: List of AnswerRelevancyVerdict"]

    Prompt2 --> LLMCall2[LLM evaluates each statement]
    LLMCall2 --> Extract2[Extract: list of verdicts with reasons]
    Extract2 --> StoreVerdicts[Store in self.verdicts]
    EmptyVerdicts --> StoreVerdicts

    %% Calculate Score
    StoreVerdicts --> CalcScore[Calculate Score]
    CalcScore --> ScoreLogic["Score Calculation:<br/>---<br/>If verdicts empty: score = 1<br/>Else:<br/>  relevant_count = verdicts != 'no'<br/>  score = relevant_count / total_verdicts<br/>If strict_mode AND score < threshold:<br/>  score = 0"]

    %% Generate Reason Detail
    ScoreLogic --> Step3[Generate Reason]
    Step3 --> CheckIncludeReason{include_reason?}
    CheckIncludeReason -->|No| NoReason[reason = None]
    CheckIncludeReason -->|Yes| CollectIrrelevant[Collect irrelevant statements<br/>where verdict = 'no']

    CollectIrrelevant --> Prompt3["<b>PROMPT 3: generate_reason</b><br/>---<br/>Input: irrelevant_statements + input + score<br/>Task: Provide concise reason for score<br/>---<br/>Template Structure:<br/>- Instructions to explain score<br/>- Why not higher?<br/>- Why at current level?<br/>- If no irrelevant statements:<br/>positive upbeat message<br/>- Multimodal rules if applicable<br/>- JSON format with 'reason' key<br/>---<br/>Output: Explanation string"]

    Prompt3 --> LLMCall3[LLM generates explanation]
    LLMCall3 --> Extract3[Extract: reason string]
    Extract3 --> StoreReason[Store in self.reason]
    NoReason --> StoreReason

    %% Final Steps
    StoreReason --> CheckSuccess[Determine success<br/>success = score >= threshold]
    CheckSuccess --> VerboseLogs[Construct verbose logs:<br/>1. Statements list<br/>2. Verdicts list<br/>3. Score + Reason]
    VerboseLogs --> PostMetric{log_metric_to<br/>_confident?}
    PostMetric -->|Yes| SendMetric[Post metric data]
    PostMetric -->|No| ReturnScore
    SendMetric --> ReturnScore([Return self.score])

    %% Styling
    classDef promptClass fill:#e1f5ff,stroke:#0066cc,stroke-width:2px
    classDef llmClass fill:#fff4e1,stroke:#ff9900,stroke-width:2px
    classDef calcClass fill:#f0e1ff,stroke:#9900cc,stroke-width:2px

    class Prompt1,Prompt2,Prompt3 promptClass
    class LLMCall1,LLMCall2,LLMCall3 llmClass
    class CalcScore,ScoreLogic calcClass

```

Answer Correctness

- involves gauging the accuracy of `the generated answer` (actual output) when compared to `the ground truth` (expected output). This evaluation relies on the `ground truth` and the `answer`, with scores ranging from 0 to 1. A higher score indicates a closer alignment between the generated answer and the ground truth, signifying better correctness.

Context Precision

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

- evaluates whether all of the ground-truth relevant items present in the `contexts` are ranked higher or not. Ideally all the relevant chunks must appear at the top ranks. This metric is computed using the `question`, `ground_truth` and the `contexts`, with values ranging between 0 and 1, where higher scores indicate better precision.

Logic Flow (mermaid)

```
flowchart TD
    Start([measure called]) --> CheckParams[Check LLM test case params<br/>INPUT, EXPECTED_OUTPUT &<br/>RETRIEVAL_CONTEXT required]
    CheckParams --> AsyncCheck{async_mode?}

    AsyncCheck -->|Yes| AsyncMeasure[Run a_measure via event loop]
    AsyncCheck -->|No| SyncFlow[Continue synchronous flow]

    SyncFlow --> Step1[Generate Verdicts]
    AsyncMeasure --> Step1A[Generate Verdicts async]

    %% Multimodal Preprocessing
    Step1 --> MultimodalCheck{multimodal?}
    Step1A --> MultimodalCheck
    MultimodalCheck -->|Yes| AnnotateContext[Annotate retrieval context<br/>with Node IDs:<br/>Node 1: context1<br/>Node 2: context2<br/>etc.]
    MultimodalCheck -->|No| DirectContext[Use retrieval context as-is]

    AnnotateContext --> Prompt1
    DirectContext --> Prompt1

    %% Generate Verdicts Detail
    Prompt1["<b>PROMPT 1: generate_verdicts</b><br/>---<br/>Input: input + expected_output + retrieval_context<br/>Task: Determine if each context node is useful<br/>---<br/>Template Structure:<br/>- Goal: Was each node useful in arriving<br/>  at the expected output?<br/>- Document count displayed<br/>- Multimodal: contexts annotated with Node IDs<br/>- Example with Einstein/Nobel Prize:<br/>  • 'Einstein won Nobel Prize' → yes<br/>  • 'Won in 1968' → yes<br/>  • 'There was a cat' → no<br/>- Verdict options: yes/no ONLY<br/>  • 'yes' = node was useful/relevant<br/>  • 'no' = node was not useful<br/>- Must provide 'reason' for BOTH verdicts<br/>- Reason should quote parts of context<br/>- JSON format with verdicts array<br/>- ONE verdict per context node<br/>---<br/>Output: List of ContextualPrecisionVerdict"]

    Prompt1 --> LLMCall1[LLM evaluates usefulness<br/>of each context node]
    LLMCall1 --> Extract1[Extract: list of verdicts with reasons]
    Extract1 --> StoreVerdicts[Store in self.verdicts]

    %% Calculate Score - Weighted Cumulative Precision
    StoreVerdicts --> CalcScore[Calculate Score]
    CalcScore --> ScoreLogic["Score Calculation (Weighted Cumulative Precision):<br/>---<br/>If verdicts empty: score = 0<br/>Else:<br/>  1. Convert verdicts to binary list:<br/>  'yes' → 1, 'no' → 0<br/>  2. For each position k (starting from 1):<br/>     If node is relevant (=1):<br/>       precision_at_k = relevant_count / k<br/>       weighted_sum += precision_at_k<br/>       relevant_count++<br/>  3. If no relevant nodes: score = 0<br/>  Else: score = weighted_sum / relevant_count<br/>---<br/>This rewards ranking relevant nodes HIGHER<br/>If strict_mode AND score < threshold:<br/>  score = 0"]

    %% Score Example Box
    ScoreLogic --> ScoreExample["<b>Score Example:</b><br/>---<br/>Context order: [yes, no, yes, yes]<br/>---<br/>k=1: relevant, precision=1/1=1.0, sum=1.0<br/>k=2: not relevant, skip<br/>k=3: relevant, precision=2/3=0.67, sum=1.67<br/>k=4: relevant, precision=3/4=0.75, sum=2.42<br/>---<br/>Score = 2.42/3 = 0.807<br/>---<br/>Better ordering [yes, yes, yes, no]:<br/>Would give score = 1.0"]

    %% Generate Reason Detail
    ScoreExample --> Step2[Generate Reason]
    Step2 --> CheckIncludeReason{include_reason?}
    CheckIncludeReason -->|No| NoReason[reason = None]
    CheckIncludeReason -->|Yes| PrepareVerdicts[Prepare verdict data:<br/>List of dicts with<br/>verdict + reason for each node]

    PrepareVerdicts --> Prompt2["<b>PROMPT 2: generate_reason</b><br/>---<br/>Input: input + verdicts + score<br/>Task: Explain the contextual precision score<br/>---<br/>Template Structure:<br/>- Context: verdicts list with verdict/reason/node<br/>- Contextual precision definition:<br/>  Measures if relevant nodes ranked<br/>  HIGHER than irrelevant nodes<br/>- Retrieval contexts given IN ORDER OF RANKINGS<br/>- Instructions:<br/>  • Explain why score not higher<br/>  • Explain why at current level<br/>  • MUST USE reasons and quotes from verdicts<br/>  • MUST reference node RANK (1st, 2nd, etc.)<br/>  • Don't mention 'verdict' - say 'irrelevant nodes'<br/>  • Don't mention 'reason fields' - just use info<br/>  • Explain why 'no' verdicts should be ranked lower<br/>  • Make explicit they are nodes in retrieval context<br/>  • If score = 1:
```

@Copyright 2025 by Vietnam Silicon

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

```
short positive upbeat message<br/>- JSON format with 'reason' key<br/>---<br/>Output: Explanation string"]

    Prompt2 --> LLMCall2[LLM generates ranking explanation]
    LLMCall2 --> Extract2[Extract: reason string]
    Extract2 --> StoreReason[Store in self.reason]
    NoReason --> StoreReason

    %% Final Steps
    StoreReason --> CheckSuccess[Determine success<br/>success = score >= threshold]
    CheckSuccess --> VerboseLogs[Construct verbose logs:<br/>1. Verdicts list<br/>2. Score + Reason]
    VerboseLogs --> PostMetric{log_metric_to<br/>_confident?}
    PostMetric -->|Yes| SendMetric[Post metric data]
    PostMetric -->|No| ReturnScore
    SendMetric --> ReturnScore([Return self.score])

    %% Key Differences Note
    KeyDiff["<b>KEY DIFFERENCES FROM OTHER METRICS:</b><br/>---<br/>• Only 2 LLM calls (not 3-4)<br/>• No separate extraction phase<br/>• Evaluates RANKING quality, not just presence<br/>• Score considers POSITION of relevant nodes<br/>• Lower score if relevant nodes ranked low<br/>• Uses weighted cumulative precision formula"]

    ReturnScore -.->|Note| KeyDiff

    %% Styling
    classDef promptClass fill:#e1f5ff,stroke:#0066cc,stroke-width:2px
    classDef llmClass fill:#fff4e1,stroke:#ff9900,stroke-width:2px
    classDef calcClass fill:#f0e1ff,stroke:#9900cc,stroke-width:2px
    classDef exampleClass fill:#ffe1e1,stroke:#cc0000,stroke-width:2px
    classDef noteClass fill:#f0f0f0,stroke:#666666,stroke-width:2px,stroke-dasharray: 5 5

    class Prompt1,Prompt2 promptClass
    class LLMCall1,LLMCall2 llmClass
    class CalcScore,ScoreLogic calcClass
    class ScoreExample exampleClass
    class KeyDiff noteClass

```

Context Recall

Context Utilization

@Copyright 2025 by Vietnam Silicon.